

Arduino Cookbook : De la Théorie à la Pratique

Chapitre 15 : Internet et Réseaux (Ethernet)

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

15.0 Introduction : L'Internet des Objets (IoT)

Connecter l'Arduino au monde entier

La communication série et la radio locale sont utiles, mais l'ajout d'une connectivité IP (Internet Protocol) permet de contrôler un système depuis l'autre bout du monde ou d'enregistrer des données dans le Cloud.

Dans ce chapitre, nous allons apprendre à :

- Utiliser le Shield Ethernet (Puce W5100 / W5500).
- Comprendre et configurer les adresses MAC et IP.
- Créer un Serveur Web local pour afficher des capteurs sur un navigateur.
- Transformer l'Arduino en Client Web pour récupérer des données en ligne.
- Utiliser le protocole UDP pour des communications rapides.

15.1 Le Matériel : Le Shield Ethernet

L'Arduino Uno n'a pas de port réseau natif. On utilise une carte d'extension (Shield) qui s'enfiche directement au-dessus.

Caractéristiques techniques :

- Puce principale : **Wiznet W5100** (ou W5500 plus récent). Elle gère la pile TCP/IP de manière matérielle, soulageant ainsi le petit processeur de l'Arduino.
- Connexion RJ45 standard pour câble réseau.
- **Protocole de communication** : Le Shield communique avec l'Arduino via le **bus SPI** (vu au Chapitre 13).
- **Broches monopolisées** : 11, 12, 13 (SPI) et la broche **10** (CS pour l'Ethernet).

15.2 Adressage Réseau : MAC et IP

Pour exister sur un réseau, l'Arduino a besoin de deux identifiants :

1. **L'Adresse MAC (Media Access Control)** : L'adresse physique unique de la carte. Sur les anciens shields, il fallait l'inventer dans le code (ex : 0xDE, 0xAD, 0xBE, 0xEF, 0xFE, 0xED).
2. **L'Adresse IP (Internet Protocol)** : L'adresse logique sur le réseau local (ex : 192.168.1.177).

DHCP vs IP Statique

Le **DHCP** permet à votre box internet d'attribuer automatiquement une adresse IP à l'Arduino. L'**IP Statique** est fixée par vous-même, ce qui est indispensable si l'Arduino doit héberger un site web (Serveur).

Code : Initialisation du réseau (DHCP)

La bibliothèque standard s'appelle `Ethernet.h`. Voici comment connecter la carte à votre réseau domestique via DHCP.

```
#include <SPI.h>
#include <Ethernet.h>

// Adresse MAC arbitraire (assurez-vous qu'elle soit unique sur votre reseau)
byte mac[] = { 0xDE, 0xAD, 0xBE, 0xEF, 0xFE, 0xED };

void setup() {
  Serial.begin(9600);

  // Tente d'obtenir une adresse IP via le routeur (DHCP)
  if (Ethernet.begin(mac) == 0) {
    Serial.println("Echec de configuration DHCP.");
    while(true); // Bloque le programme en cas d'erreur
  }

  Serial.print("Connecte ! Mon adresse IP est : ");
  Serial.println(Ethernet.localIP());
}

void loop() {}
```

15.3 Architecture Client vs Serveur

Il est crucial de comprendre la différence entre ces deux rôles :

Rôle	Le Serveur (Server)	Le Client (Client)
Action Principale	Écoute et attend les requêtes.	Initie la connexion et demande.
Exemple Web	Le site <code>google.com</code> .	Votre navigateur web (Chrome).
Cas d'usage Arduino	Afficher la température de la maison sur une page web.	Envoyer un tweet ou interroger une API Météo.

15.4 L'Arduino en tant que Serveur Web

Si l'on veut lire des données de capteurs depuis son smartphone en tapant l'adresse IP de l'Arduino dans le navigateur, il faut utiliser la classe `EthernetServer`.

Le processus :

1. On démarre le serveur sur le port **80** (le port standard pour le protocole HTTP).
2. Dans la boucle `loop()`, on écoute si un client s'est connecté.
3. Si oui, on lit sa requête.
4. On lui renvoie du code HTML, ligne par ligne.
5. On ferme la connexion.

Code : Serveur Web Minimal (Partie 1)

```
#include <SPI.h>
#include <Ethernet.h>

byte mac[] = { 0xDE, 0xAD, 0xBE, 0xEF, 0xFE, 0xED };
IPAddress ip(192, 168, 1, 177); // Adresse IP Statique !
EthernetServer server(80);      // Port HTTP standard

void setup() {
  Ethernet.begin(mac, ip);
  server.begin(); // Demarre le serveur
  Serial.println("Serveur Web actif.");
}
```


Code : Serveur Web Minimal (Partie 2 - HTML)

```
void loop() {
  EthernetClient client = server.available(); // Y a-t-il un visiteur ?
  if (client) {
    boolean ligneVide = true; // Permet de detecter la fin de la requete HTTP
    while (client.connected()) {
      if (client.available()) {
        char c = client.read();

        // La requete HTTP se termine toujours par une ligne vide (\r\n\r\n)
        if (c == '\n' && ligneVide) {
          // En-tete HTTP standard
          client.println("HTTP/1.1 200 OK");
          client.println("Content-Type: text/html");
          client.println("Connection: close");
          client.println();

          // Corps de la page HTML (Utilisez F() pour economiser la RAM !)
          client.println("<!DOCTYPE HTML><html>");
          client.print("Valeur du capteur : ");
          client.print(analogRead(A0));
          client.println("</html>");
          break; // Sort de la boucle
        }
        else if (c != '\r') { ligneVide = false; }
      }
    }
  }
}
```

15.5 Interaction Web : Allumer une LED

Comment l'utilisateur peut-il agir sur l'Arduino depuis la page Web ?

Il faut ajouter des liens ou des boutons HTML qui modifient l'URL.

- Si l'utilisateur clique sur "Allumer", le navigateur envoie une requête GET vers l'adresse : `http://192.168.1.177/?led=on`
- L'Arduino doit analyser le texte brut de la requête entrante via `client.read()`.
- S'il détecte la chaîne de caractères `?led=on`, il exécute un `digitalWrite(PIN, HIGH)`.

C'est une excellente façon de créer une petite interface domotique locale.

15.6 L'Arduino en tant que Client Web

L'inverse : L'Arduino va chercher des informations sur le web de manière autonome.

Le processus avec EthernetClient :

1. Demander au routeur l'adresse IP du site voulu (Résolution DNS).
2. Se connecter au serveur sur le port 80.
3. Envoyer une requête HTTP GET (ex : "Donne-moi la page /api/meteo.txt").
4. Lire la réponse du serveur et isoler la donnée utile.

Code : Requête Client (Récupérer un fichier)

```
EthernetClient client;
char serveur[] = "www.monsite.com"; // Nom de domaine

void setup() {
  Ethernet.begin(mac); // DHCP

  if (client.connect(serveur, 80)) {
    // 1. Envoi de la requete GET
    client.println("GET /fichier.txt HTTP/1.1");
    client.println("Host: www.monsite.com");
    client.println("Connection: close");
    client.println(); // Ligne vide obligatoire !
  }
}

void loop() {
  // 2. Lecture de la reponse
  if (client.available()) {
    char c = client.read();
    Serial.print(c); // Affiche le contenu reçu sur le moniteur serie
  }

  if (!client.connected()) {
    // 3. Fermeture de la connexion manuellement
  }
}
```

15.7 Extraire des données complexes (Parsing)

Lorsqu'un Arduino interroge une API Météo (ex : OpenWeatherMap), le serveur répond avec beaucoup de texte, souvent au format **JSON** ou **XML**.

Le défi de la mémoire : L'Arduino Uno (2 Ko de RAM) **ne peut pas** stocker la page entière pour l'analyser !

Solutions :

- Lire le flux à la volée (octet par octet) et ignorer tout jusqu'à trouver un mot-clé précis (ex : "temp":).
- Utiliser des bibliothèques légères comme `ArduinoJson` conçues spécifiquement pour analyser ces structures sans saturer la mémoire.

15.8 Le Protocole UDP

Jusqu'à présent, nous parlions de **TCP** (utilisé par HTTP), qui est un protocole "fiable" : il s'assure que le destinataire a bien reçu chaque paquet de données.

Le protocole UDP (User Datagram Protocol) : Il envoie les données dans le réseau sans vérifier si elles arrivent à destination ("Fire and Forget").

Avantages :

- Beaucoup plus rapide et léger que le TCP.
- Idéal pour envoyer des données de capteurs en temps réel à 50 Hz sur un réseau local.
- Indispensable pour interroger des serveurs de temps (NTP).

15.9 Cas Pratique UDP : Serveur de Temps (NTP)

Au Chapitre 12, nous avons utilisé un module RTC (DS3231) pour garder l'heure. Avec un Shield Ethernet, le RTC devient optionnel !

- L'Arduino envoie un petit paquet UDP vers un serveur NTP (ex : `time.nist.gov` sur le port 123).
- Le serveur répond instantanément avec l'heure universelle exacte (le nombre de secondes écoulées depuis 1900).
- On combine cela avec la bibliothèque `TimeLib.h` pour synchroniser l'horloge interne de l'Arduino automatiquement une fois par jour.

Résumé du Chapitre 15

- **SPI et Pin 10** : N'oubliez pas que le Shield Ethernet utilise les broches SPI. La broche 10 ne doit pas être utilisée pour autre chose.
- **Serveur vs RAM** : Écrire une belle page HTML demande beaucoup de texte. Utilisez impérativement la macro `F()` : `client.println(F("<html>"))`; pour sauver votre RAM (SRAM).
- **Le parsing à la volée** : C'est la compétence la plus difficile mais la plus nécessaire pour créer des Clients Web efficaces sur de petits microcontrôleurs.
- **Sécurité** : Un petit Arduino n'est pas conçu pour gérer du HTTPS (cryptage SSL/TLS) complexe à cause de son manque de puissance de calcul. Ne faites pas transiter de mots de passe sensibles.

Fin du Chapitre 15. Le Chapitre 16 abordera la gestion, l'utilisation et la création de Bibliothèques (Libraries).